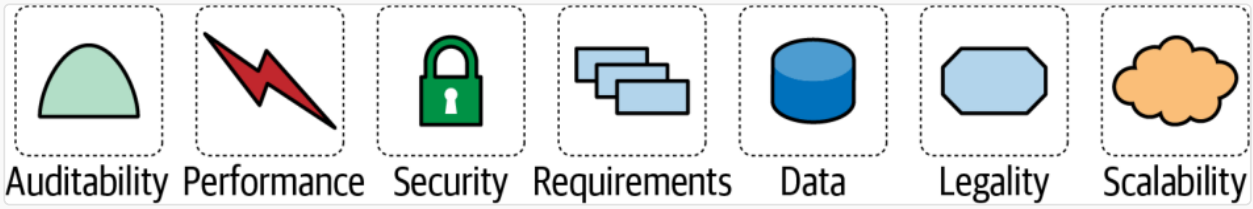
فصل ۴: تعریف ویژگی‌های معماری

اکنون به جزئیاتِ طراحی ساختاری `(structural design)` می‌پردازیم که یکی از نقش‌های کلیدیِ معمارانِ نرم‌افزار است. این کار عمدتاً از دو فعالیت تشکیل می‌شود: تحلیلِ ویژگی‌های معماری، که در این فصل به آن می‌پردازیم، و طراحیِ مؤلفه‌های منطقی، که در فصل ۸ بررسی می‌شود. معماران می‌توانند این دو فعالیت را به هر ترتیبی (یا حتی به‌موازات هم) انجام دهند، اما این دو در یک نقطهٔ اتصال `(join point)` حیاتی به هم می‌رسند.

وقتی شرکتی تصمیم می‌گیرد مسئلهٔ معینی را با نرم‌افزار حل کند، فهرستی از الزاماتِ آن سیستم گرد می‌آورد (برای استخراجِ این الزامات فنونِ بسیاری وجود دارد که در فصل ۸ به آن‌ها پرداخته‌ایم). در سراسرِ کتاب، از این الزامات با عنوانِ حوزهٔ مسئله `(problem domain)` —یا فقط «حوزه»— یاد می‌کنیم. در فصل ۱ دیدید که ویژگی‌های معماری آن جنبه‌های مهمی از سیستم‌اند که از حوزهٔ مسئله مستقل‌اند و برای موفقیتِ سیستم اهمیت دارند. در این فصل، تعریفِ این اصطلاح و نیز ویژگی‌های معماریِ مشخص را عمیق‌تر بررسی می‌کنیم.

معماران اغلب در تعریفِ حوزه همکاری می‌کنند، اما همچنین باید همهٔ کارهایی را که نرم‌افزار باید انجام دهد و مستقیماً به کارکردهای حوزه مربوط نیست، تعریف، کشف و به هر شکلِ دیگر تحلیل کنند: همان ویژگی‌های معماری. نقشِ معماران در تعریفِ ویژگی‌های معماری بخشی از چیزی است که معماری نرم‌افزار را از کدنویسی و طراحی متمایز می‌کند. آنان همچنین باید هنگامِ طراحیِ یک راهکارِ نرم‌افزاری عواملِ بسیارِ دیگری را در نظر بگیرند، همان‌طور که در شکل ۴-۱ نشان داده شده است.



شکل ۴-۱. یک راهکارِ نرم‌افزاری هم از الزاماتِ حوزه تشکیل می‌شود و هم از ویژگی‌های معماری.

ماندگاریِ اصطلاحِ «الزاماتِ غیرکارکردی»

بسیاری از سازمان‌ها ویژگی‌های معماری را با اصطلاحاتِ گوناگونی توصیف می‌کنند، از جمله الزاماتِ غیرکارکردی (non-functional requirements)؛ اصطلاحی که برای تمایزِ ویژگی‌های معماری از الزاماتِ کارکردی (functional requirements) ساخته شد. ما این اصطلاح را نمی‌پسندیم، چون بار معنایی منفی دارد و اهمیت موضوع را دست‌کم می‌گیرد: چگونه می‌توان تیم‌ها را قانع کرد که به چیزی «غیرکارکردی» به‌اندازهٔ کافی توجه کنند؟ اصطلاحِ رایجِ دیگر صفاتِ کیفی (quality attributes) است که آن را هم نمی‌پسندیم، چون بیشتر ارزیابیِ کیفیت پس از وقوع را القا می‌کند تا طراحی را.

ما اصطلاحِ ویژگی‌های معماری را ترجیح می‌دهیم، چون دغدغه‌هایی را توصیف می‌کند که برای موفقیتِ معماری —و در نتیجه کلِ سیستم— حیاتی‌اند، بی‌آنکه از اهمیتِ این دغدغه‌ها بکاهد. در کتابِ Head First Software Architecture (O'Reilly، ۲۰۲۴)، از ویژگی‌های معماری با عنوانِ قابلیت‌های (capabilities) سیستم یاد می‌کنیم؛ در مقابل، حوزه نمایانگرِ رفتارِ (behavior) سیستم است.

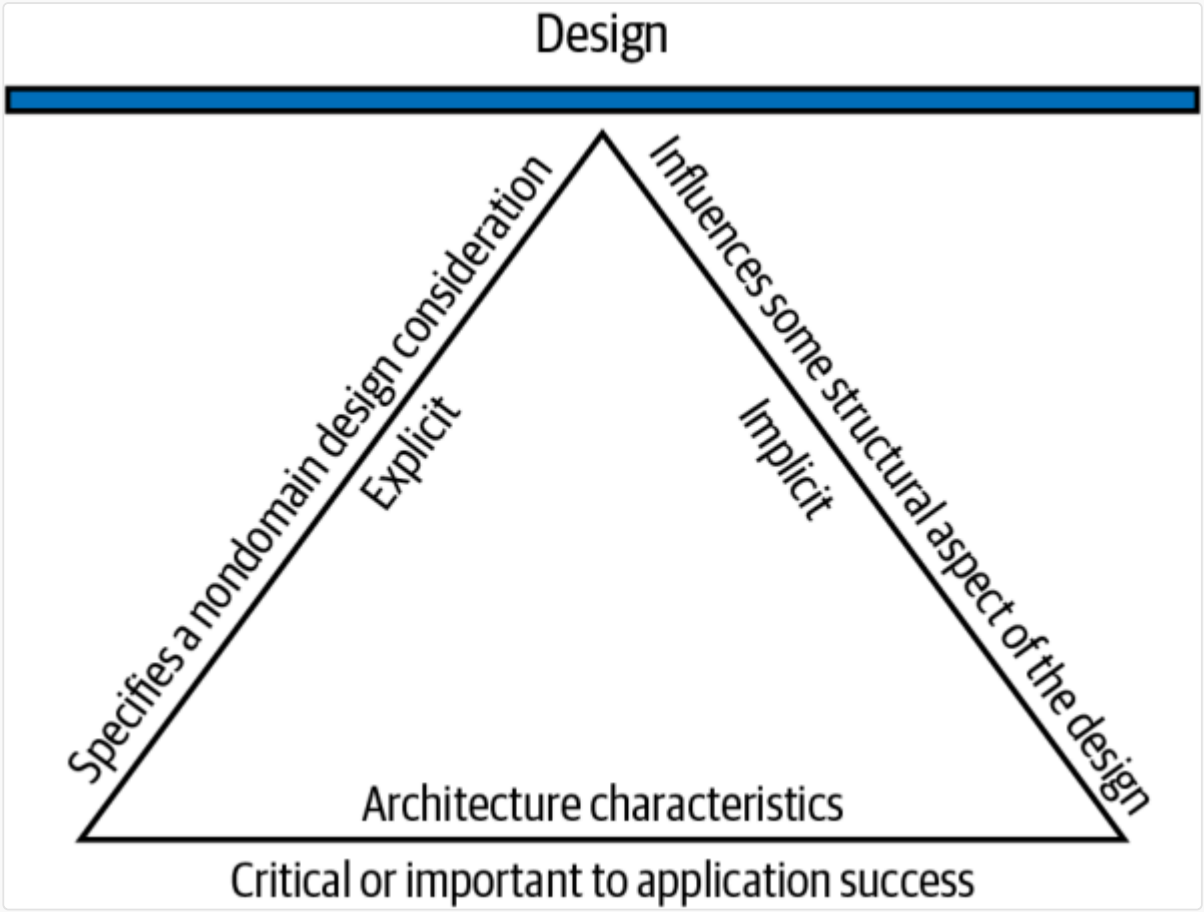
گاهی اصطلاحات «گیر می‌کنند»، و الزاماتِ غیرکارکردی ظاهراً در میانِ معمارانِ نرم‌افزار از آن اصطلاحاتِ بسیار چسبنده است. این اصطلاح هنوز در بسیاری از سازمان‌ها رایج است. نخستین بار در اواخرِ دههٔ ۱۹۷۰ در ادبیاتِ مهندسیِ نرم‌افزار پیدا شد، تقریباً هم‌زمان با تحلیلِ نقاطِ کارکردی (function point analysis): فنی برای برآورد که الزاماتِ سیستم را به «نقاطِ کارکردی» تجزیه می‌کند و هر نقطه نمایانگرِ واحدی از کار است. از لحاظِ نظری، تیم‌ها می‌توانستند در پایانِ فرایندِ تحلیل همهٔ نقاطِ کارکردی را جمع بزنند و دیدی نسبت به پروژه به دست آورند. متأسفانه این روش فقط لایه‌ای ظاهری از قطعیت فراهم می‌کرد، اما مانندِ بسیاری از طرح‌های برآورد گرفتارِ ذهنی‌گرایی بود و دیگر به کار نمی‌رود.

با این حال، یک بینش از آن دوران با ما مانده است: بخشِ بزرگی از تلاشی که برای ساختنِ یک سیستم صرف می‌شود، به قابلیت‌های آن سیستم مربوط است و نه به الزاماتش. آنان این تلاش‌ها را نقاطِ غیرکارکردی (non-function points) نامیدند و همین باعث شد اصطلاحِ الزاماتِ غیرکارکردی رواج یابد.

ویژگی‌های معماری و طراحیِ سیستم

یک الزام برای آنکه ویژگیِ معماری به شمار آید، باید سه ملاک را برآورده کند: باید یک ملاحظهٔ طراحیِ غیرحوزه‌ای (nondomain) را مشخص سازد، بر جنبه‌ای ساختاری از طراحی اثر بگذارد، و برای موفقیتِ برنامه حیاتی یا مهم باشد. این اجزای درهم‌تنیدهٔ تعریفِ ما در شکل ۴-۲ نشان داده شده‌اند که از همین سه جزء و چند قیدِ تکمیلی تشکیل شده

است.



شکل ۴-۲. ویژگی‌های متمایزکنندهٔ ویژگی‌های معماری

بیایید این اجزا را دقیق‌تر بررسی کنیم:

ویژگیِ معماری، یک ملاحظهٔ طراحیِ غیرحوزه‌ای را مشخص می‌کند.

طراحیِ ساختاری در معماری نرم‌افزار از دو فعالیتِ معمار تشکیل می‌شود: درکِ حوزهٔ مسئله و کشفِ اینکه سیستم برای موفقیت به پشتیبانی از چه نوع قابلیت‌هایی نیاز دارد. ملاحظاتِ طراحیِ حوزه رفتارِ سیستم را پوشش می‌دهند و ویژگی‌های معماری قابلیت‌ها را تعریف می‌کنند. این دو فعالیت در کنار یکدیگر، شالودهٔ طراحیِ ساختاری را شکل می‌دهند.

در حالی که الزاماتِ طراحی مشخص می‌کنند برنامه چه کاری باید انجام دهد، ویژگی‌های معماری مشخص می‌کنند الزامات چگونه پیاده‌سازی شوند و چرا انتخاب‌های معینی صورت گرفته است؛ خلاصه آنکه، ملاک‌های عملیاتی و طراحی برای موفقیتِ پروژه را تعیین می‌کنند.

برای نمونه، سطوحِ مشخصی از کارایی اغلب ویژگیِ معماریِ مهمی است، اما اغلب در اسنادِ الزامات نمی‌آید. نکتهٔ مرتبط‌تر اینکه هیچ سندِ الزاماتی عملاً نمی‌گوید طراحی باید «از بدهی فنی جلوگیری کند»، اما این یک ملاحظهٔ طراحیِ رایج است. این تمایزِ میانِ ویژگی‌های صریح و نامشهود را در «استخراجِ ویژگی‌های معماری از دغدغه‌های حوزه»

(صفحهٔ ۶۷) به‌تفصیل بررسی می‌کنیم.

ویژگیِ معماری بر جنبه‌ای ساختاری از طراحی اثر می‌گذارد.

دلیلِ اصلیِ اینکه معماران می‌کوشند ویژگی‌های معماریِ پروژه‌ها را توصیف کنند، بیرون‌کشیدنِ ملاحظاتِ مهمِ طراحی است. آیا معمار می‌تواند آن را از راهِ طراحی پیاده کند، یا این ویژگیِ معماری برای موفقیت به ملاحظهٔ ساختاریِ ویژه‌ای نیاز دارد؟

برای نمونه، امنیت (security) تقریباً در هر پروژه‌ای دغدغه است و همهٔ سیستم‌ها باید در طراحی و کدنویسی برخی احتیاط‌های پایه را رعایت کنند. با این حال، امنیت زمانی به سطحِ یک ویژگیِ معماری می‌رسد که معمار تشخیص دهد معماری برای پشتیبانی از آن به ساختارِ ویژه‌ای نیاز دارد.

دو ویژگیِ معماریِ رایج را در نظر بگیرید: امنیت و مقیاس‌پذیری. معماران می‌توانند در یک سیستمِ تک‌پارچه با رعایتِ بهداشتِ خوبِ کدنویسی (coding hygiene)، از جمله فنونِ شناخته‌شده‌ای مانندِ رمزنگاری (encryption)، درهم‌سازی (hashing) و نمک‌زنی (salting)، امنیت را تأمین کنند. (توابع برازشِ معماری، که آن‌ها نیز زیرِ همین چتر قرار می‌گیرند، در فصل ۶ بررسی می‌شوند.) در مقابل، در یک معماریِ توزیع‌شده (distributed architecture) مانندِ میکروسرویس‌ها، معمار سرویسی مقاوم‌تر با پروتکل‌های دسترسیِ سخت‌گیرانه‌تر می‌سازد: رویکردی ساختاری. بنابراین، معماران می‌توانند امنیت را یا از راهِ طراحی تأمین کنند یا از راهِ ساختار. از سوی دیگر، مقیاس‌پذیری را در نظر بگیرید: هیچ اندازه از طراحیِ هوشمندانه نمی‌تواند به یک معماریِ تک‌پارچه امکان دهد از نقطه‌ای معین فراتر مقیاس‌پذیر شود. فراتر از آن نقطه، سیستم باید به سبکِ معماریِ توزیع‌شده تغییر یابد.

معماران به ویژگی‌های عملیاتیِ معماری (operational architectural characteristics) توجهِ ویژه‌ای دارند (که در «ویژگی‌های عملیاتیِ معماری» در صفحهٔ ۵۹ بررسی می‌شوند)، چون این‌ها ویژگی‌هایی هستند که بیش از همه به پشتیبانیِ ساختاریِ ویژه نیاز دارند.

ویژگیِ معماری باید برای موفقیتِ برنامه حیاتی یا مهم باشد.

برنامه‌ها می‌توانند از شمارِ عظیمی از ویژگی‌های معماری پشتیبانی کنند... اما نباید چنین کنند. هر ویژگیِ معماری‌ای که سیستم از آن پشتیبانی می‌کند، بر پیچیدگیِ طراحی‌اش می‌افزاید. به همین دلیل معماران باید بکوشند کمترین شمارِ ممکن از ویژگی‌های معماری را برگزینند، نه بیشترین را.

ما ویژگی‌های معماری را به ویژگی‌های نامشهود و صریح (explicit) تقسیم می‌کنیم. ویژگی‌های نامشهود به‌ندرت در الزامات می‌آیند، اما برای موفقیتِ پروژه ضروری‌اند. در دسترس‌بودن، قابلیتِ اطمینان (reliability) و امنیت زیربنای تقریباً همهٔ برنامه‌ها هستند، اما به‌ندرت در اسنادِ طراحی تصریح می‌شوند. معماران باید از دانشِ خود دربارهٔ حوزهٔ مسئله بهره بگیرند تا این ویژگی‌های معماری را در مرحلهٔ تحلیل کشف کنند. برای نمونه، یک شرکتِ معاملاتِ پربسامد (high-frequency trading) شاید نیازی نداشته باشد تأخیرِ (latency) پایین را در تک‌تکِ سیستم‌ها تصریح کند، چون معمارانِ آن حوزهٔ مسئله از پیش می‌دانند این ویژگی چقدر حیاتی است. ویژگی‌های صریحِ معماری در اسنادِ الزامات یا دیگر دستورالعمل‌های مشخص می‌آیند.

در شکل ۴-۲، انتخابِ مثلث عامدانه است: هر یک از اجزای تعریف از بقیه پشتیبانی می‌کند و آن‌ها نیز به نوبهٔ خود از طراحیِ کلیِ سیستم پشتیبانی می‌کنند. تکیه‌گاهی که مثلث پدید می‌آورد نشان می‌دهد این ویژگی‌های معماری اغلب چگونه بر یکدیگر اثر می‌گذارند. به همین دلیل است که معماران این‌قدر از اصطلاحِ مصالحه استفاده می‌کنند.

ویژگی‌های معماری (فهرستی ناقص)

ویژگی‌های معماری در طیفی گسترده از پیچیدگی قرار می‌گیرند؛ از ویژگی‌های سطحِ پایینِ کد (مانند ماژولاریتی) تا دغدغه‌های عملیاتیِ پیچیده (مانند مقیاس‌پذیری و کشسانی). هیچ استانداردِ جهان‌شمولِ واقعی‌ای وجود ندارد، هرچند کسانی کوشیده‌اند آن را مدون کنند. در عوض، هر سازمان این اصطلاحات را به شیوهٔ خود تفسیر می‌کند. افزون بر این، چون زیست‌بومِ نرم‌افزار بسیار سریع تغییر می‌کند، مفاهیم، اصطلاحات، سنجه‌ها و روش‌های راستی‌آزماییِ تازه پیوسته پدید می‌آیند و فرصت‌های تازه‌ای برای تعریفِ ویژگی‌های معماری فراهم می‌کنند.

با اینکه حجم و گسترهٔ عظیمِ ویژگی‌های معماری کمّی‌کردنِ آن‌ها را دشوار می‌کند، معماران آن‌ها را دسته‌بندی می‌کنند. بخش‌های پیش رو چند دستهٔ کلیِ این‌چنینی را توصیف می‌کنند و نمونه‌هایی به دست می‌دهند.

ویژگی‌های عملیاتیِ معماری

ویژگی‌های عملیاتیِ معماری قابلیت‌هایی مانند کارایی، مقیاس‌پذیری، کشسانی، در دسترس‌بودن و قابلیتِ اطمینان را پوشش می‌دهند. جدول ۴-۱ برخی از ویژگی‌های عملیاتیِ معماری را فهرست می‌کند.

جدول ۴-۱. ویژگی‌های عملیاتیِ رایجِ معماری

اصطلاح	تعریف
در دسترس‌بودن (Availability)	سیستم چه مدت از زمان باید در دسترس باشد؛ اگر شبانه‌روزی و همیشگی (24/7) باشد، باید تدابیری در کار باشد تا سیستم در صورتِ هر خرابی به‌سرعت دوباره راه بیفتد و کار کند.
تداوم (Continuity)	قابلیتِ بازیابیِ سیستم پس از فاجعه.
کارایی (Performance)	سیستم چقدر خوب عمل می‌کند؛ راه‌های سنجشِ آن شاملِ آزمونِ فشار، تحلیلِ اوج، تحلیلِ بسامدِ کارکردهای مورد استفاده و زمان‌های پاسخ است.
بازیابی‌پذیری (Recoverability)	الزاماتِ تداومِ کسب‌وکار: در صورتِ وقوعِ فاجعه، سیستم با چه سرعتی باید دوباره برخط شود. این شاملِ راهبردهای پشتیبان‌گیری و الزاماتِ سخت‌افزارِ تکراری است.
قابلیتِ اطمینان/ایمنی (Reliability/safety)	آیا سیستم باید ایمن در برابرِ خرابی (fail-safe) باشد، یا به گونه‌ای حیاتی است که بر جانِ انسان‌ها اثر می‌گذارد؟ اگر از کار بیفتد، آیا هزینهٔ هنگفتی برای شرکت در پی دارد؟ این اغلب یک طیف است، نه یک دوگانهٔ صفر و یک.
استواری (Robustness)	توانِ سیستم در مدیریتِ خطاها و شرایطِ مرزی در حینِ اجرا؛ برای نمونه، اگر اتصالِ اینترنت یا برق قطع شود.
مقیاس‌پذیری (Scalability)	توانِ سیستم در اجرا و عملکرد با افزایشِ شمارِ کاربران یا درخواست‌ها.

ویژگی‌های عملیاتیِ معماری همپوشانیِ زیادی با دغدغه‌های عملیات و DevOps دارند.

ویژگی‌های ساختاریِ معماری

معماران مسئولِ ساختارِ درستِ کدند. در بسیاری از موارد، معمار مسئولیتِ انحصاری یا مشترکِ کیفیتِ کد را بر عهده دارد، از جمله ماژولاریتیِ آن، خوانایی‌اش، میزانِ مهارِ کوپلینگ میانِ مؤلفه‌ها، تمیزیِ کد و انبوهی از ارزیابی‌های کیفیِ درونیِ دیگر. جدول ۴-۲ چند ویژگیِ ساختاریِ معماری (structural architectural characteristics) را فهرست می‌کند.

جدول ۴-۲. ویژگی‌های ساختاریِ معماری

اصطلاح	تعریف
پیکربندی‌پذیری (Configurability)	کاربرانِ نهایی تا چه اندازه آسان می‌توانند جنبه‌هایی از پیکربندیِ نرم‌افزار را از راهِ رابط‌ها تغییر دهند.
گسترش‌پذیری (Extensibility)	معماری تا چه اندازه تغییراتی را که کارکردِ موجودش را گسترش می‌دهند در خود جای می‌دهد.
نصب‌پذیری (Installability)	نصبِ سیستم روی همهٔ پلتفرم‌های لازم چقدر آسان است.
بهره‌گیریِ مجدد/بازاستفاده (Leverageability/reuse)	میزانی که می‌توان از مؤلفه‌های مشترکِ سیستم در چندین محصول بهره گرفت.
بومی‌سازی (Localization)	پشتیبانی از چند زبان در صفحه‌های ورود/پرس‌وجو و در فیلدهای داده.
قابلیتِ نگهداری (Maintainability)	اعمالِ تغییرات و بهبودِ سیستم چقدر آسان است.
قابلیتِ حمل (Portability)	توانِ سیستم در اجرا روی بیش از یک پلتفرم (مانند Oracle و SAP DB).
ارتقاپذیری (Upgradeability)	ارتقا به نسخهٔ جدیدتر روی سرورها و کلاینت‌ها چقدر آسان و سریع است.

ویژگی‌های ابری

زیست‌بومِ توسعهٔ نرم‌افزار پیوسته تغییر می‌کند و تکامل می‌یابد؛ تازه‌ترین و بارزترین نمونه‌اش ورودِ ابر است. هنگامِ انتشارِ ویراستِ نخست، رایانشِ ابری وجود داشت اما فراگیر نبود. اکنون بیشترِ سیستم‌ها دست‌کم در حدی با سیستم‌های مبتنی بر ابر تعامل دارند. چند مورد از این ملاحظات در جدول ۴-۳ آمده است.

جدول ۴-۳. ویژگی‌های معماریِ مرتبط با ارائه‌دهندهٔ ابر

اصطلاح	تعریف
مقیاس‌پذیریِ بر حسبِ تقاضا (On-demand scalability)	توانِ ارائه‌دهندهٔ ابر در افزایشِ پویای منابع بر پایهٔ تقاضا.
کشسانیِ بر حسبِ تقاضا (On-demand elasticity)	انعطاف‌پذیریِ ارائه‌دهندهٔ ابر هنگامِ جهشِ ناگهانیِ تقاضا برای منابع؛ مشابهِ مقیاس‌پذیری.
در دسترس‌بودنِ مبتنی بر ناحیه (Zone-based availability)	توانِ ارائه‌دهندهٔ ابر در جداسازیِ منابع بر حسبِ ناحیه‌های محاسباتی برای ساختنِ سیستم‌های تاب‌آورتر.
حریمِ خصوصی و امنیتِ مبتنی بر منطقه (Region-based privacy and security)	توانِ قانونیِ ارائه‌دهندهٔ ابر برای ذخیره‌سازیِ داده‌های کشورها و مناطقِ گوناگون. بسیاری از کشورها قوانینی دارند که تعیین می‌کند داده‌های شهروندانشان کجا می‌تواند نگهداری شود (و اغلب ذخیره‌سازی بیرون از منطقهٔ خود را محدود می‌کنند).

در ویراستِ دومِ این کتاب، به هر یک از فصل‌های سبک‌های معماری بخشی افزوده‌ایم که توصیف می‌کند آن سبک چگونه ملاحظاتِ ابری را در خود جای می‌دهد و تسهیل می‌کند.

ویژگی‌های میان‌بُرشیِ معماری

با اینکه بسیاری از ویژگی‌های معماری در دسته‌هایی به‌آسانی قابلِ شناسایی جای می‌گیرند، برخی دیگر بیرون از این دسته‌ها قرار می‌گیرند یا به دسته‌بندی تن نمی‌دهند، اما همچنان قیدها و ملاحظاتِ طراحیِ مهمی پدید می‌آورند. جدول ۴-۴ چند مورد از این ویژگی‌های میان‌بُرشی (cross-cutting) را توصیف می‌کند.

جدول ۴-۴. ویژگی‌های میان‌بُرشیِ معماری

اصطلاح	تعریف
دسترس‌پذیری `(Accessibility)`	همهٔ کاربران، از جمله افرادِ دارای معلولیت‌هایی مانندِ کوررنگی یا کم‌شنوایی، تا چه اندازه آسان می‌توانند به سیستم دسترسی داشته باشند.
بایگانی‌پذیری `(Archivability)`	قیدهای سیستم دربارهٔ بایگانی یا حذفِ داده‌ها پس از گذشتِ دوره‌ای معین.
احرازِ هویت `(Authentication)`	الزاماتِ امنیتی برای اطمینان از اینکه کاربران همان کسی هستند که ادعا می‌کنند.
مجوزدهی `(Authorization)`	الزاماتِ امنیتی برای اطمینان از اینکه کاربران فقط به کارکردهای معینی در برنامه دسترسی دارند (بر حسبِ مورد کاربرد، زیرسیستم، صفحهٔ وب، قاعدهٔ کسب‌وکار، سطحِ فیلد و مانند آن).
الزاماتِ حقوقی `(Legal)`	قیدهای قانونی‌ای که سیستم در چارچوبِ آن‌ها کار می‌کند؛ مانندِ قوانینِ حفاظت از داده همچون `GDPR`، یا قوانینِ سوابقِ مالی همچون `Sarbanes-Oxley` در ایالات متحده، یا هر مقرراتی دربارهٔ شیوهٔ ساخت یا استقرارِ برنامه. این شاملِ حقوقِ محفوظی است که شرکت لازم می‌داند.
حریمِ خصوصی `(Privacy)`	توانِ سیستم در رمزگذاری و پنهان‌کردنِ تراکنش‌ها از کارکنانِ داخلیِ شرکت، حتی مدیرانِ پایگاه‌داده `(DBA)` و معمارانِ شبکه.
امنیت `(Security)`	قواعد و قیدهای مربوط به رمزگذاری در پایگاه‌داده یا در ارتباطاتِ شبکه‌ای میانِ سیستم‌های داخلی؛ احرازِ هویت برای دسترسیِ کاربرانِ راه دور، و دیگر تدابیرِ امنیتی.
پشتیبانی‌پذیری `(Supportability)`	سطحِ پشتیبانیِ فنی‌ای که برنامه نیاز دارد؛ اینکه تا چه حد ثبتِ وقایع `(logging)` و امکاناتِ دیگر برای اشکال‌زداییِ خطاهای سیستم لازم است.
کاربردپذیری/دستیابی‌پذیری `(Usability/achievability)`	سطحِ آموزشی که کاربران برای دستیابی به اهدافشان با برنامه/راهکار نیاز دارند.

هر فهرستی از ویژگی‌های معماری ناگزیر ناقص خواهد بود؛ هر پروژهٔ نرم‌افزاری ممکن است بر پایهٔ عواملِ خاصِ خود ویژگی‌های معماریِ تازه‌ای ابداع کند. بسیاری از اصطلاحاتی که برشمردیم نادقیق و مبهم‌اند، گاهی به سببِ ظرافت‌های معنایی و گاهی به سببِ نبودِ تعریف‌های عینی. برای نمونه، همکنش‌پذیری `(interoperability)` و سازگاری

(compatibility) ممکن است هم‌ارز به نظر برسند و در برخی سیستم‌ها چنین هم باشد. با این حال، این دو با هم فرق دارند، چون همکنش‌پذیری بر سهولتِ یکپارچه‌شدن با سیستم‌های دیگر دلالت دارد که خود مستلزمِ API های منتشرشده و مستند است. در مقابل، سازگاری بیشتر به استانداردهای صنعت و حوزه مربوط است. نمونهٔ دیگر یادگیری‌پذیری (learnability) است: یک تعریف آن را «اینکه کاربران چقدر آسان می‌توانند استفاده از نرم‌افزار را یاد بگیرند» می‌داند و تعریفی دیگر «سطحی که سیستم می‌تواند به‌طورِ خودکار دربارهٔ محیطش یاد بگیرد تا با بهره‌گیری از الگوریتم‌های یادگیری ماشین، خودپیکربند یا خودبهینه‌ساز شود».

بسیاری از تعریف‌ها با هم همپوشانی دارند؛ برای نمونه، در دسترس‌بودن و قابلیتِ اطمینان. پروتکلِ اینترنت (IP) را در نظر بگیرید که زیربنای TCP است. IP در دسترس است اما قابلِ اطمینان نیست: بسته‌ها ممکن است نامرتب برسند و گیرنده ممکن است ناچار شود بسته‌های گم‌شده را دوباره درخواست کند.

هیچ فهرستِ کاملی از استانداردها برای تعریفِ این دسته‌ها وجود ندارد. International Organization for Standards (ISO) فهرستی منتشر می‌کند که بر پایهٔ قابلیت‌ها سازمان یافته و با فهرستِ ما همپوشانی دارد، اما عمدتاً فهرستی ناقص از دسته‌ها را پایه می‌گذارد. در ادامه برخی از تعریف‌های ISO آمده است که برای به‌روزکردنِ اصطلاحات و افزودنِ دسته‌هایی هم‌راستا با دغدغه‌های مدرن، بازنویسی شده‌اند:

کارایی و بهره‌وری (Performance efficiency): سنجشِ کارایی نسبت به میزانِ منابعِ مصرف‌شده در شرایطِ معلوم. این شاملِ رفتارِ زمانی (time behavior)، یعنی سنجشِ زمان‌های پاسخ و پردازش و/یا نرخِ توانِ عملیاتی؛ بهره‌برداری از منابع (resource utilization)، یعنی میزان و نوعِ منابعِ مصرف‌شده؛ و ظرفیت (capacity)، یعنی میزانِ فراتررفتن از حدودِ بیشینهٔ تعیین‌شده، است.

سازگاری: میزانی که یک محصول، سیستم یا مؤلفه می‌تواند با محصولات، سیستم‌ها یا مؤلفه‌های دیگر اطلاعات مبادله کند و/یا کارکردهای لازمش را در حالی انجام دهد که همان محیطِ سخت‌افزاری یا نرم‌افزاری را با دیگران به اشتراک گذاشته است. این شاملِ همزیستی (coexistence)، یعنی توانِ انجامِ کارآمدِ کارکردهای لازم در حالِ اشتراکِ محیط و منابع با محصولاتِ دیگر؛ و همکنش‌پذیری، یعنی میزانی که دو یا چند سیستم می‌توانند اطلاعات را مبادله و از آن استفاده کنند، است.

کاربردپذیری (Usability): کاربران می‌توانند سیستم را به‌شکلی اثربخش، کارآمد و رضایت‌بخش برای هدفِ مورد نظرش به کار گیرند. این شاملِ تشخیص‌پذیریِ تناسب (appropriateness recognizability)، یعنی کاربران بتوانند تشخیص دهند که نرم‌افزار برای نیازهایشان مناسب است یا نه؛ یادگیری‌پذیری، یعنی کاربران چقدر آسان می‌توانند استفاده از نرم‌افزار را یاد بگیرند؛ محافظت در برابرِ خطای کاربر (user error protection)؛ و دسترس‌پذیری، یعنی در دسترس قراردادنِ نرم‌افزار برای افرادی با گسترده‌ترین طیفِ ویژگی‌ها و توانایی‌ها، است.

قابلیتِ اطمینان: میزانی که سیستم در شرایطِ مشخص و برای دوره‌ای مشخص کار می‌کند. این ویژگی زیردسته‌هایی مانندِ این‌ها را در بر می‌گیرد: بلوغ (maturity)، یعنی آیا نرم‌افزار در کارکردِ عادی نیازهای قابلیتِ اطمینان را برآورده می‌کند؛ در دسترس‌بودن، یعنی نرم‌افزار عملیاتی و قابلِ دسترسی است؛ تحملِ خطا، یعنی آیا نرم‌افزار به‌رغمِ خطاهای

سخت‌افزاری یا نرم‌افزاری همان‌طور که باید کار می‌کند؛ و بازیابی‌پذیری، یعنی آیا نرم‌افزار می‌تواند با بازیابیِ داده‌های آسیب‌دیده از خرابی بازیابی شود و وضعیتِ مطلوبِ سیستم را دوباره برقرار کند.

امنیت: میزانی که نرم‌افزار از اطلاعات و داده‌ها محافظت می‌کند، به‌طوری که افراد یا محصولات یا سیستم‌های دیگر سطحی از دسترسی به داده داشته باشند که با نوع و سطحِ مجوزشان متناسب است. این خانواده از ویژگی‌ها شاملِ محرمانگی `(confidentiality)`، یعنی داده فقط برای کسانی قابلِ دسترسی است که مجوزِ دسترسی دارند؛ یکپارچگی `(integrity)`، یعنی نرم‌افزار از دسترسی یا تغییرِ غیرمجاز در نرم‌افزار یا داده جلوگیری می‌کند؛ انکارناپذیری `(nonrepudiation)`، یعنی آیا می‌توان ثابت کرد که کنش‌ها یا رویدادها رخ داده‌اند؛ پاسخ‌گویی‌پذیری `(accountability)`، یعنی آیا می‌توان کنش‌های یک کاربر را ردیابی کرد؛ و اصالت `(authenticity)`، یعنی اثباتِ هویتِ کاربر، است.

قابلیتِ نگهداری: نمایانگرِ میزانِ اثربخشی و کارآمدی‌ای است که توسعه‌دهندگان با آن می‌توانند نرم‌افزار را برای بهبود، اصلاح یا انطباق با تغییراتِ محیط و/یا الزامات تغییر دهند. این ویژگی شاملِ ماژولاریتی، یعنی میزانی که نرم‌افزار از مؤلفه‌های مجزا تشکیل شده است؛ بازاستفاده‌پذیری `(reusability)`، یعنی میزانی که توسعه‌دهندگان می‌توانند یک دارایی را در بیش از یک سیستم یا در ساختِ دارایی‌های دیگر به کار گیرند؛ تحلیل‌پذیری `(analyzability)`، یعنی توسعه‌دهندگان چقدر آسان می‌توانند معیارهای عینی دربارهٔ نرم‌افزار گرد آورند؛ اصلاح‌پذیری `(modifiability)`، یعنی میزانی که توسعه‌دهندگان می‌توانند نرم‌افزار را بدونِ ایجادِ نقص یا کاهشِ کیفیتِ محصولِ موجود تغییر دهند؛ و آزمون‌پذیری `(testability)`، یعنی توسعه‌دهندگان و دیگران چقدر آسان می‌توانند نرم‌افزار را بیازمایند، است.

قابلیتِ حمل: میزانی که توسعه‌دهندگان می‌توانند یک سیستم، محصول یا مؤلفه را از یک محیطِ سخت‌افزاری، نرم‌افزاری یا محیطِ عملیاتی یا کاربریِ دیگر به محیطی دیگر منتقل کنند. این ویژگی زیرویژگی‌های انطباق‌پذیری `(adaptability)`، یعنی آیا توسعه‌دهندگان می‌توانند نرم‌افزار را به‌شکلی اثربخش و کارآمد برای محیط‌های سخت‌افزاری، نرم‌افزاری یا عملیاتی و کاربریِ متفاوت یا در حالِ تحول انطباق دهند؛ نصب‌پذیری، یعنی آیا نرم‌افزار را می‌توان در محیطی مشخص نصب و/یا حذف کرد؛ و جایگزین‌پذیری `(replaceability)`، یعنی توسعه‌دهندگان چقدر آسان می‌توانند کارکرد را با نرم‌افزاری دیگر جایگزین کنند، را در بر می‌گیرد.

آخرین موردِ فهرستِ `ISO` به جنبه‌های کارکردیِ نرم‌افزار می‌پردازد:

تناسبِ کارکردی `(Functional suitability)`: این ویژگی نمایانگرِ میزانی است که یک محصول یا سیستم، هنگامِ استفاده در شرایطِ مشخص، کارکردهایی فراهم می‌کند که نیازهای تصریح‌شده و ضمنی را برآورده می‌کنند. این ویژگی از زیرویژگی‌های زیر تشکیل شده است:

- **کامل‌بودنِ کارکردی `(Functional completeness)`:** میزانی که مجموعهٔ کارکردها همهٔ وظایف و اهدافِ مشخص‌شدهٔ کاربر را پوشش می‌دهد.
- **درستیِ کارکردی `(Functional correctness)`:** میزانی که یک محصول یا سیستم نتایجِ درست را با درجهٔ دقتِ لازم فراهم می‌کند.

- **مناسب‌بودنِ کارکردی** `(Functional appropriateness)` : میزانی که کارکردها انجامِ وظایف و اهدافِ مشخص را تسهیل می‌کنند.

با این حال، ما باور نداریم که تناسبِ کارکردی به این فهرست تعلق داشته باشد. این مورد، ویژگی‌های معماری را توصیف نمی‌کند، بلکه الزاماتِ انگیزه‌بخش برای ساختنِ نرم‌افزار را بیان می‌دارد. این نشان می‌دهد که اندیشیدن دربارهٔ رابطهٔ میانِ ویژگی‌های معماری و حوزهٔ مسئله چگونه تکامل یافته است. این تکامل را در فصل ۷ بررسی می‌کنیم.

> **ابهام‌های فراوان در معماری نرم‌افزار**
>
> یکی از منابعِ همیشگیِ دلسردیِ معماران، نبودِ تعریف‌های روشن برای این‌همه چیزِ حیاتی است، از جمله خودِ فعالیتِ معماری نرم‌افزار! نبودِ استاندارد باعث می‌شود شرکت‌ها برای چیزهای رایج اصطلاحاتِ خاصِ خود را تعریف کنند. این امر اغلب به سردرگمی در سراسرِ صنعت می‌انجامد، چون معماران یا اصطلاحاتِ نامفهوم به کار می‌برند یا، بدتر از آن، اصطلاحاتِ یکسان را برای معانیِ به‌کلی متفاوت.
>
> هر قدر هم که بخواهیم، نمی‌توانیم واژگانِ استانداردی را بر دنیای توسعهٔ نرم‌افزار تحمیل کنیم. با این حال، طراحیِ حوزه‌محور برای کمک به پرهیز از سوءتفاهم‌های ناشی از اصطلاحات، به سازمان‌ها توصیه می‌کند زبانی فراگیر `(ubiquitous language)` میانِ کارکنان برقرار کنند و به کار گیرند. ما نیز از این توصیه پیروی می‌کنیم و آن را توصیه می‌کنیم.

مصالحه‌ها و کم‌بدترین معماری

پیش‌تر گفتیم که معماران باید فقط از آن ویژگی‌های معماری پشتیبانی کنند که برای موفقیتِ سیستم حیاتی یا مهم‌اند. سیستم‌ها به دلایلِ گوناگون تنها می‌توانند از چندتایی از ویژگی‌های معماری‌ای که برشمردیم پشتیبانی کنند. نخست، پشتیبانی از یک ویژگیِ معماری به‌ندرت بدون هزینه است. هر ویژگیِ پشتیبانی‌شده به تلاشِ طراحی از سوی معمار، تلاشِ توسعه‌دهندگان برای پیاده‌سازی و نگهداری‌اش، و شاید پشتیبانیِ ساختاری نیاز دارد.

دوم، ویژگی‌های معماری با یکدیگر و با حوزهٔ مسئله هم‌افزا `(synergistic)` هستند. هر قدر هم که چیزِ دیگری را ترجیح دهیم، هر عنصرِ طراحی با همهٔ عناصرِ دیگر برهم‌کنش دارد. برای نمونه، برداشتنِ گام‌هایی برای بهبودِ امنیت تقریباً به‌یقین بر کارایی اثرِ منفی می‌گذارد: برنامه باید رمزگذاریِ درجا، غیرمستقیم‌سازی (`indirection`، برای پنهان‌کردنِ اسرار) و فعالیت‌های دیگری انجام دهد که می‌توانند کارایی را کاهش دهند. خلبانانِ هواپیما اغلب هنگامِ یادگیریِ هدایتِ بالگرد دچار چالش می‌شوند، چرا که بالگرد برای هر دست و هر پا یک کنترلرِ جداگانه دارد. تغییرِ یک کنترل بر همهٔ کنترل‌های دیگر اثر می‌گذارد، چون آن‌ها هم‌افزا هستند. هدایتِ بالگرد تمرینی مداوم در برقراریِ توازن

است و این تمثیل به‌خوبی فرایندِ مصالحه در انتخابِ ویژگی‌های معماری را توصیف می‌کند؛ این ویژگی‌ها نیز به همین شکل هم با دیگر ویژگی‌های معماری و هم با طراحیِ حوزه هم‌افزا هستند: تغییرِ یکی اغلب تغییرِ دیگری را در پی دارد. معماران نیز، درست مانندِ خلبانِ بالگرد، باید بیاموزند که این اجزای درهم‌تنیده را هم‌زمان مدیریت کنند.

سوم، همان‌طور که پیش‌تر گفتیم، نبودِ تعریف‌های استاندارد برای ویژگی‌های معماری باعث می‌شود سازمان‌ها با ابهام دست‌وپنجه نرم کنند. هرچند صنعت هرگز نخواهد توانست فهرستی تغییرناپذیر از ویژگی‌های معماری بسازد (چون ویژگی‌های تازه پیوسته پدید می‌آیند)، هر سازمان می‌تواند فهرستِ خود (یا زبانِ فراگیرِ خود) را با تعریف‌های عینی بسازد.

سرانجام، نه‌تنها شمارِ ویژگی‌های معماری پیوسته رو به افزایش است، بلکه شمارِ دسته‌ها نیز در دههٔ گذشته افزایش یافته است. برای نمونه، چند دهه پیش، معماران چندان به دغدغه‌های عملیاتی اهمیت نمی‌دادند و آن‌ها را «جعبهٔ سیاهی» جداگانه می‌دانستند. با این حال، با محبوبیتِ روزافزونِ معماری‌هایی مانندِ میکروسرویس‌ها، معماران و تیم‌های عملیات باید فشرده‌تر و پرتکرارتر همکاری کنند. هرچه معماری نرم‌افزار پیچیده‌تر می‌شود، پیوندِ نزدیک‌تری با بخش‌های دیگرِ سازمان برقرار می‌کند.

بنابراین، بسیار به‌ندرت پیش می‌آید که معماران بتوانند سیستمی طراحی کنند و تک‌تکِ ویژگی‌های معماری را به حداکثر برسانند. بیشتر اوقات، تصمیم‌ها به مصالحه میانِ چند دغدغهٔ رقیب ختم می‌شوند.

> **نکته:** هرگز برای بهترین معماری تلاش نکنید؛ کم‌بدترین معماری (least worst architecture) را هدف بگیرید.

تلاش برای پشتیبانی از ویژگی‌های معماریِ بیش از اندازه، به راهکارهای عمومی‌ای می‌انجامد که می‌کوشند همهٔ مسائلِ کسب‌وکار را حل کنند. چنین طراحی‌ای به‌سرعت دست‌وپاگیر می‌شود و از همین رو این معماری‌ها به‌ندرت کار می‌کنند.

بکوشید معماری‌ای طراحی کنید که تا جای ممکن تکاملی و تکرارپذیر باشد. هرچه تغییرِ معماری آسان‌تر باشد، نگرانی از بابتِ دستیابی به گزینه‌ای بی‌نقص در همان گامِ نخست کمتر خواهد بود. یکی از مهم‌ترین درس‌های توسعهٔ نرم‌افزارِ چابک (Agile) ارزشِ تکرار است؛ این اصل در همهٔ سطوحِ توسعهٔ نرم‌افزار، از جمله معماری، صادق است.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Structural Design	طراحی ساختاری
Join Point	نقطهٔ اتصال
Problem Domain	حوزهٔ مسئله

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Non-Functional Requirements	الزامات غیرکارکردی
Functional Requirements	الزامات کارکردی
Quality Attributes	صفات کیفی
Capabilities	قابلیت‌ها
Behavior	رفتار
Function Point Analysis	تحلیل نقاط کارکردی
Non-Function Points	نقاط غیرکارکردی
Nondomain	غیرحوزه‌ای
Security	امنیت
Coding Hygiene	بهداشت کدنویسی
Encryption	رمزنگاری
Hashing	درهم‌سازی
Salting	نمک‌زنی
Distributed Architecture	معماری توزیع‌شده
Operational Architectural Characteristics	ویژگی‌های عملیاتی معماری
Explicit Architecture Characteristic	ویژگی صریح معماری
Reliability	قابلیت اطمینان
High-Frequency Trading	معاملات پربسامد
Latency	تأخیر
Structural Architectural Characteristics	ویژگی‌های ساختاری معماری
Continuity	تداوم
Recoverability	بازیابی‌پذیری
Reliability/Safety	قابلیت اطمینان/ایمنی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Fail-safe	ایمن در برابر خرابی
Robustness	استواری
Configurability	پیکربندی‌پذیری
Extensibility	گسترش‌پذیری
Installability	نصب‌پذیری
Leverageability/Reuse	بهره‌گیری مجدد/بازاستفاده
Localization	بومی‌سازی
Portability	قابلیت حمل
Upgradeability	ارتقاپذیری
On-demand Scalability	مقیاس‌پذیری بر حسب تقاضا
On-demand Elasticity	کشسانی بر حسب تقاضا
Zone-based Availability	در دسترس‌بودن مبتنی بر ناحیه
Region-based Privacy and Security	حریم خصوصی و امنیت مبتنی بر منطقه
Cross-Cutting Architectural Characteristics	ویژگی‌های میان‌بُرشی معماری
Accessibility	دسترس‌پذیری
Archivability	بایگانی‌پذیری
Authentication	احراز هویت
Authorization	مجوزدهی
Legal	الزامات حقوقی
Privacy	حریم خصوصی
Supportability	پشتیبانی‌پذیری
Usability/Achievability	کاربردپذیری/دستیابی‌پذیری
Logging	ثبت وقایع

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Interoperability	همکنش‌پذیری
Compatibility	سازگاری
Learnability	یادگیری‌پذیری
Performance Efficiency	کارایی و بهره‌وری
Time Behavior	رفتار زمانی
Resource Utilization	بهره‌برداری از منابع
Capacity	ظرفیت
Coexistence	همزیستی
Usability	کاربردپذیری
Appropriateness Recognizability	تشخیص‌پذیری تناسب
User Error Protection	محافظت در برابر خطای کاربر
Maturity	بلوغ
Confidentiality	محرمانگی
Integrity	یکپارچگی
Nonrepudiation	انکارناپذیری
Accountability	پاسخ‌گویی‌پذیری
Authenticity	اصالت
Reusability	بازاستفاده‌پذیری
Analyzability	تحلیل‌پذیری
Modifiability	اصلاح‌پذیری
Testability	آزمون‌پذیری
Adaptability	انطباق‌پذیری
Replaceability	جایگزین‌پذیری

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Functional Suitability	تناسب کارکردی
Functional Completeness	کامل‌بودن کارکردی
Functional Correctness	درستی کارکردی
Functional Appropriateness	مناسب‌بودن کارکردی
Ubiquitous Language	زبان فراگیر
Synergistic	هم‌افزا
Indirection	غیرمستقیم‌سازی
Least Worst Architecture	کم‌بدترین معماری
Agile	چابک